Circuits & QNodes

What is PennyLane?

PennyLane is a free, community-driven, open-source library.

It lets us build and test quantum programs, including hybrid programs that mix quantum and classical computing.

We can run these programs on:

- CPU
- GPU
- Quantum simulators
- Real quantum hardware

It was first creating for quantum machine learning, but today it is used across many areas of quantum computing research.

Prerequisites

Math background:

- Linear algebra
- Complex numbers
- Multi-variable calculus

Python skills:

- Basic python programming
- Variables, lists
- Conditional statements and loops
- Functions
- Using NumPy

Quantum Computing basics:

- Quantum states as vector in a Hilbert space
- Unitary operators like Hadamard, Pauli gates, rotations, and CNOT
- Controlled gate
- Quantum measurements and observables
- Born’s rule; how probabilities are calculated
- Expectation values
- How quantum circuits represent state preparation, operations, and measurements

---

Installing & Importing

Installation:

```
pip install pennylane
```

Importing:

```
import pennylane as qml
from pennylane import numpy as np
```

Quantum functions

In PennyLane, a quantum circuit, aka quantum function, is just a Python function that:

- Applied quantum gates
- Optionally depends on parameters
- Acts on specific wires (qubits)

All wires start in the default state $|0\rangle$.

Example: A simple structure

```
def quantum_function(parameters):
    qml.gate1(parameters, wires=...)
    qml.gate2(wires=...)
```

This means:

i. Apply gate 1

ii. Then apply gate 2

A concrete example:

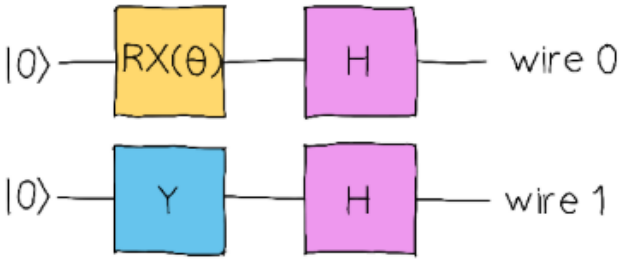
Here's a small circuit using:

- qml.RX (rotation around X-axis)
- qml.Pauliy
- qml.Hadamard

```
def my_first_quantum_function(theta):
    qml.RX(theta, wires=0)
    qml.PauliY(wires=1)
    qml.Hadamard(wires=0)
    qml.Hadamard(wires=1)
```

Here:

- qml.RX(theta, wires=0) ⇒ Rotates qubit 0 by angle theta.
- qml.PauliY(wires=1) ⇒ Applies Pauli-Y gate to qubit 1.
- qml.Hadamard(wires=0), qml.Hadamard(wires=1) ⇒ Applies a Hadamard gates to both qubits.



Understanding wires (qubits)

- Wires are labeled with integers → 0, 1, 2, ...

- Counting starts at 0
- In this example, we use two qubits ⇒ 0, 1
- By default:
  - Wire 0 is drawn at the top
  - Wire 1 is below it

Why doesn't this return anything?

- There's no "return" statement
- The function is not yet connected to a quantum device
- Needs additional setup to execute the circuit and measure something

---

Devices

To run "my_first_quantum_function", we need to return something.

One option is *qml.state()*, which gives us the quantum state as a complex array after all the gates have been applied:

```
def my_first_quantum_function(theta):
    qml.RX(theta, wires=0)
    qml.PauliY(wires=1)
    qml.Hadamard(wires=0)
    qml.Hadamard(wires=1)

    return qml.state()
```

This does not run yet.

- In PennyLane, a quantum function needs a device to run on. A device is the "hardware" that executes the qubit. There are a few types:
  1. **default.qubit** → Basic noiseless qubit simulator. Easy to use and always up-to-date.
  2. **lightning.qubit** → Fast simulator using C++. Optimized for speed, but features may lag a bit.
  3. **default.mixed** → Simulator that includes noise in gates.

- The most common one is **default.qubit**.
- For big circuits with many qubits, **lightning.qubit** is faster.

Defining a device:

```
dev = qml.device(”default.qubit”, wires=2)
```

- “wires = 2” ⇒ Tells PennyLane we have two qubits.
- For **default.qubit**, this is optional as it can figure it out from the circuit.

QNodes

A QNode is what you get when you connect a quantum function (circuit) to a device. It’s like giving the circuit a place to run.

The easiest way ⇒ @qml.qnode decorator

- We can attach a device to a quantum function using @qml.qnode:

```
dev = qml.device(”default.qubit”, wires=2)

@qml.qnode(dev)
def my_first_quantum_function(theta):
    qml.RX(theta, wires=0)
    qml.PauliY(wires=1)
    qml.Hadamard(wires=0)
    qml.Hadamard(wires=1)

    return qml.state()
```

- @qml.qnode(dev) ⇒ Turns the function into a QNode
- Running the QNode gives the quantum state

```
import numpy as np

print(my_first_qunatum_function(np.pi/4))
```

Output:

```
[ 0.191+0.462j -0.191-0.462j -0.191+0.462j  0.191-0.462j ]
```

    - These numbers are the amplitudes of the quantum state in the computational basis.
    - The order follows binary counting of the qubits: |00⟩, |01⟩, |10⟩, |11⟩

Alternative way ⇒ qml.QNode

- We can also create a QNode explicitly instead of using the decorator;

```
dev = qml.device("default.qubit", wires=2)

def my_first_quantum_function(theta):
    qml.RX(theta, wires=0)
    qml.PauliY(wires=1)
    qml.Hadamard(wires=0)
    qml.Hadamard(wires=1)

    return qml.state()

my_first_qnode = qml.QNode(my_first_quantum_function, dev)
```

- my_first_quantum_function = Just a plain quantum function
- my_first_qnode = QNode that can run on the device

```
print(my_first_qnode(np.pi/4))
```

Output:

```
[ 0.191+0.462j -0.191-0.462j -0.191+0.462j  0.191-0.462j ]
```

Custom Wires

Wires = Just labels for qubits in a quantum circuit, default numbered 0, 1, 2, ...

Specifying the number of wires

- We can tell PennyLane how many qubits the device has:

```
dev = qml.device("default.qubit", wires=3)
```

— This creates 3 qubits; 0, 1, 2

— The computational basis corresponds to these wires in order.

- We can also define them explicitly in a list or range:

```
dev = qml.device(”default.qubit”, wires=[0, 1, 2])
```

```
dev = qml.device(”default.qubit”, wires=range(3))
```

Changing the order of wires

- We can scramble the order as per requirement:

```
dev_scrambled = qml.device(”default.qubit”, wires=[2, 0, 1])
```

— Now wire 2 is treated as the first qubit, wire 0 as second, and wire 1 as third.

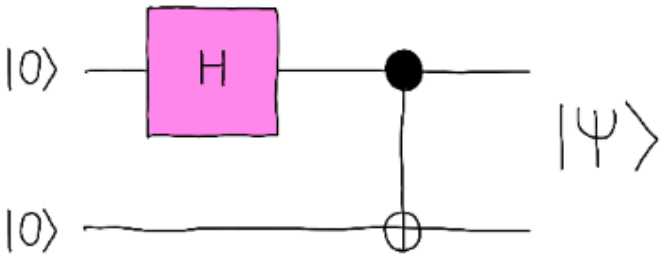
Naming wires

- We can give custom names to the wires instead of numbers

```
dev_c_t = qml.device(”default.qubit”, wires=[”target”, “control”])
```

— This is useful for readability.

Example ⇒ Creating a maximally entangled state (Bell state)




```
@qml.qnode(dev_c_t)
def create_entangled():
    qml.Hadamard(wires=”control”)
    qml.CNOT(wires=[”control”, “target”])
    return qml.state()
```

— For qml.CNOT(wires=[”control”, “target”]):

- “control” ⇒ control output
- “target” ⇒ target output

Quantum functions as sub-circuits

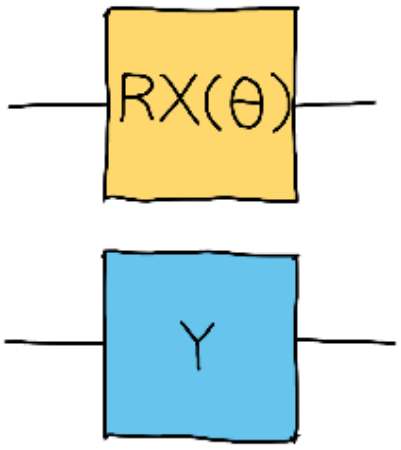
Why make quantum functions separately?

- Sometimes we want a reusable pieces of circuits.
- Those small pieces can be combined into a bigger circuits.
- Quantum functions don’t need to return anything to be used as sub-circuits.

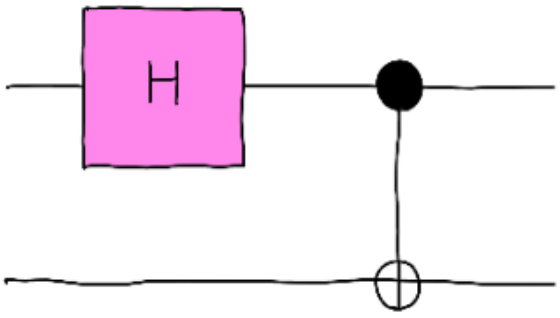
Example: Small sub-circuits

```
# First small sub-circuit
def subcircuit_1(angle):
    qml.RX(angle, wires=0)
    qml.PauliY(wires=1)

# Another small sub-circuit
def subcircuit_2():
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])
```

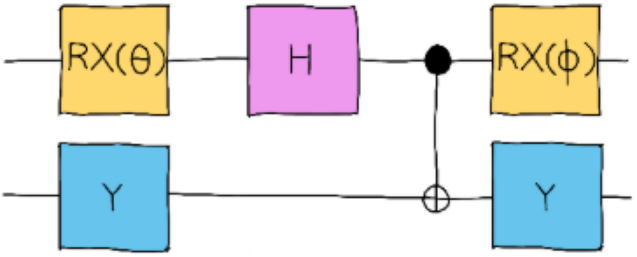
— First sub-circuit:



— Second sub-circuit:



Combining them into a full circuit




```
def full_circuit(theta, phi):
    subcircuit_1(theta)
    subcircuit_2()
    subcircuit_1(phi)
```

- full_circuit ⇒ Calls the smaller subcircuit in order
- theta, phi ⇒ Parameters that control rotations in the subcircuits

Visualizing the circuit

- “qml.draw()” can be used

```
theta = 0.3
phi = 0.2

print(qml.draw(full_circuit)(theta, phi))
```

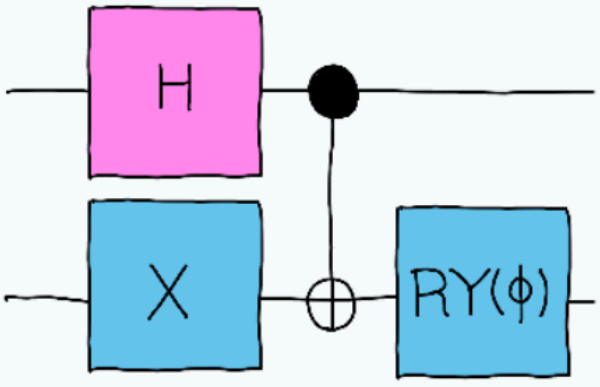
- Output:

```
0: ──RX(0.30)──H─ ╭●──RX(0.20)─┤
```

```
1: ──Y──────────────── ╰X──Y────────────┤
```

Codercise PF.1.1a - Writing a quantum function

Consider the following quantum circuit



Complete the `circuit` function that represents this quantum circuit. You will need to write the gates contained in this circuit. Note that the RY gate depends on an angle parameter!

```python
def circuit():
    # Apply the Hadamard gate to wire 0
    qml.Hadamard(wires=0)

    # Apply the PauliX gate to wire 1
    qml.PauliX(wires=1)

    # Apply the CNOT gate with wire 0 as control and wire 1 as target
    qml.CNOT(wires=[0, 1])

    # Apply the RY rotation to wire 1 with the parameter 'phi'
    qml.RY(phi, wires=1)

    return qml.state()

```

Codercise PF.1.1b - Creating documents

In the code below, define three devices as follows:

- `dev_qubit`: A `"default.qubit"` device with wires named `"alice"` and `"bob"`.
- `dev_mixed`: A `"default.mixed"` device with two wires, with no specific names.

Then, turn `example_circuit` into a QNode by choosing a device and inserting the wire names corresponding to the device of your choice. Feel free to switch devices so you can see how the output changes!

```
dev_qubit = qml.device("default.qubit", wires=["alice", "bob"]) # Define the device here
dev_mixed = qml.device("default.mixed", wires=2) # Define the device here

@qml.qnode(dev_mixed) # Choose the device you want
def example_circuit(theta):

    qml.RX(theta, wires = 0) # Complete with wires in the device
    qml.CNOT(wires = [0, 1]) # Complete with wires in the device

    return qml.state()

print(example_circuit(0.3))
```

Codercise PF.1.1c - Circuit into QNode

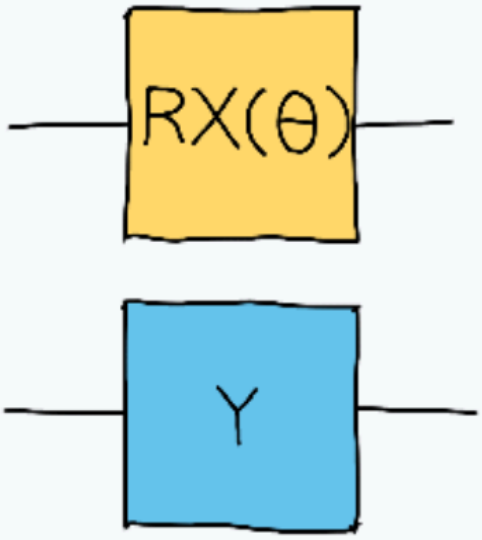
Consider the `circuit` that you defined in Codercise PF.1.1a. Define a two-wire `"default.qubit"` device and use it to define a QNode named `circuit_qnode` that applies the circuit and returns the state. The most straightforward way to do this is using the qml.QNode function.

Note: You can call `circuit` without writing it again, since it's already defined for you in the backend.

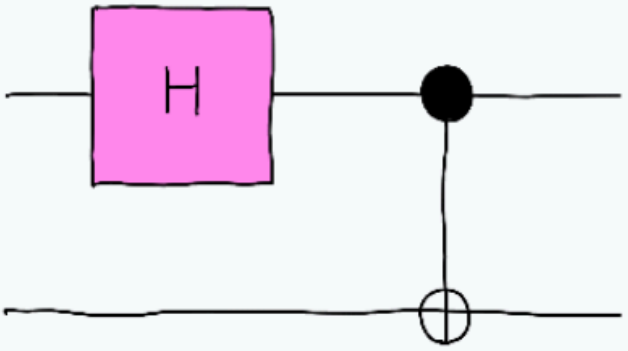
```
dev = qml.device("default.qubit", wires=2)# Define the device

circuit_qnode = qml.QNode(circuit, dev)# Assign a QNode to circuit"

print(circuit_qnode(0.3))

```

Codercise PF.1.2a - Defining subcircuits

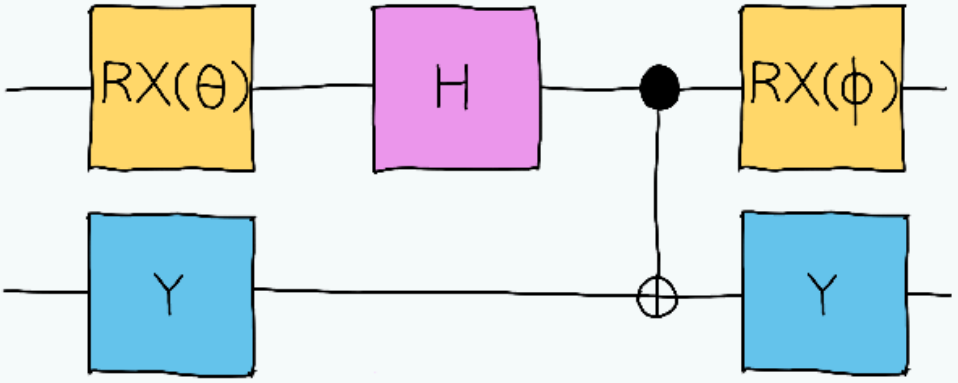
In this codercise, we will define the quantum circuits `subcircuit_1` (see below)



and `subcircuit_2` (see below).



Then, use these two subcircuits to create the `full_circuit` QNode shown below

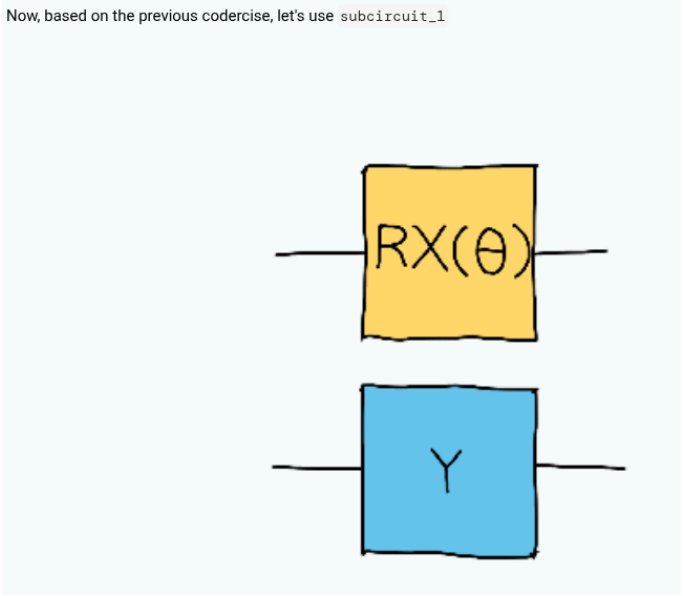


The necessary device `dev` is already given to you in the template.

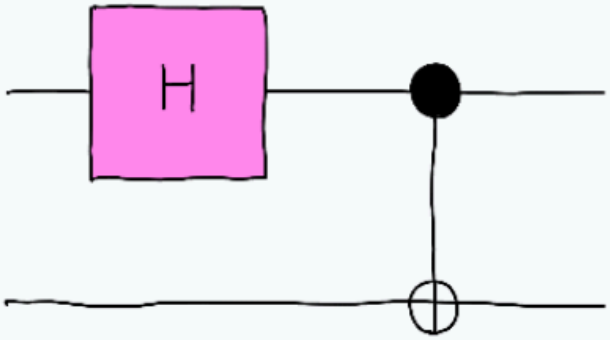
```
def subcircuit_1(angle):
    qml.RX(angle, wires=0)
    qml.PauliY(wires=1)

def subcircuit_2():
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])

dev = qml.device('default.qubit', wires = 2)

# Decorate this function to create a QNode
def full_circuit(theta, phi):
    subcircuit_1(theta)
    subcircuit_2()
    subcircuit_1(phi)
```

Codercise PF.1.2b - Jumbled wires

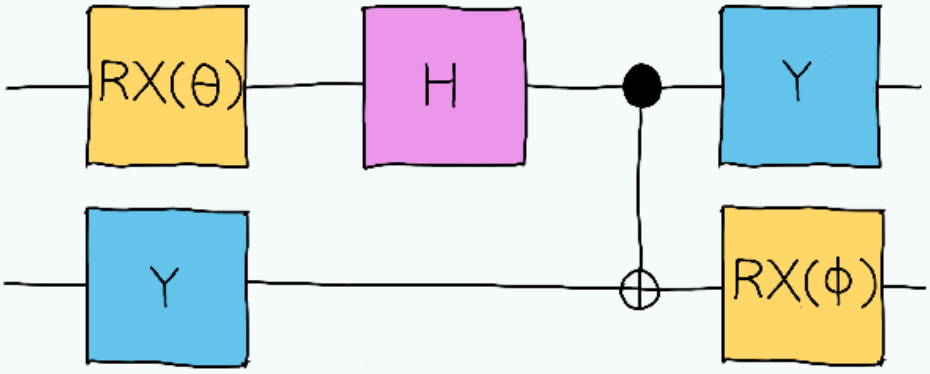
Now, based on the previous codercise, let's use `subcircuit_1`



and `subcircuit_2`



to build the following `full_circuit`.



One way to do this is to let the subcircuits depend on a list `wire_list` of wires, so that we ca switch the order of the wires when we call the subcircuits. Build the QNode `full_circuit` using this strategy.

```
def subcircuit_1(angle, wire_list):
    qml.RX(angle, wires=wire_list[0])
    qml.PauliY(wires=wire_list[1])

def subcircuit_2(wire_list):
    qml.Hadamard(wires=wire_list[0])
    qml.CNOT(wires=[wire_list[0], wire_list[1]])

dev = qml.device("default.qubit", wires = [0,1])

@qml.qnode(dev)
def full_circuit(theta, phi):
    # RX(theta) on wire 0, Y on wire 1
    subcircuit_1(theta, [0, 1])

    # H on wire 0, CNOT from 0 to 1
    subcircuit_2([0, 1])

    # Y on wire 0, RX(phi) on wire 1
    # Swap the list to [1, 0] so RX goes to wire 1 and Y goes to wire 0
    subcircuit_1(phi, [1, 0])

    return qml.state()

```